Introduction

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using objects and classes.

Python supports OOP principles, making it a powerful and flexible language for software development.

Key Principles of OOP

1. **Encapsulation** - Bundling data and methods that operate on the data within a single unit (class).
2. **Abstraction** - Hiding complex implementation details and exposing only necessary parts of an object.
3. **Inheritance** - Creating new classes that derive properties and behaviors from existing classes.
4. **Polymorphism** - Allowing objects to be treated as instances of their parent class, enabling flexibility and code reuse.

Encapsulation in Python

Encapsulation helps restrict direct access to an object's data, promoting data security.

```python
class Car:
 def __init__(self, brand, speed):
 self.brand = brand # Public attribute
 self.__speed = speed # Private attribute (encapsulation)

 def get_speed(self):
 return self.__speed # Accessing private attribute
```

Abstraction in Python

Abstraction allows defining methods in a base class without implementing them, enforcing that subclasses provide specific behavior.

```python
```

```
from abc import ABC, abstractmethod
class Animal(ABC):
 @abstractmethod
 def make_sound(self):
 pass
class Dog(Animal):
 def make_sound(self):
 return "Bark"
```

Inheritance in Python

Inheritance allows a class to derive properties and behaviors from another class, avoiding code duplication.

```python
class Vehicle:
 def __init__(self, brand):
 self.brand = brand
 def show_brand(self):
 print("Brand:", self.brand)
class Car(Vehicle):
 def __init__(self, brand, model):
 super().__init__(brand)
 self.model = model
```

Polymorphism in Python

Polymorphism enables using a unified interface to interact with different data types.

```python
class Bird:
 def sound(self):
 return "Chirp"
class Cat:
```

```
 def sound(self):
 return "Meow"
def make_sound(animal):
 print(animal.sound())
make_sound(Bird()) # Output: Chirp
make_sound(Cat()) # Output: Meow
```

Conclusion

- Encapsulation ensures data hiding and access control.
- Abstraction promotes implementation hiding.
- Inheritance facilitates code reuse.
- Polymorphism enhances flexibility in object behavior.